

Linguagens de Programação, Compiladores e Interpretadores

Níveis de linguagem de programação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

- Linguagens de programação são ferramentas básicas para todos programadores
 - Notação formal para expressar algoritmos
- Algoritmo é um conceito abstrato e existe independentemente de qualquer notação particular
 - No entanto, sem notação não se pode expressar o algoritmo, nem comunicá-lo a outros

Algoritmo 1 Troca de pneu do carro.

- 1: desligar o carro
- 2: pegar as ferramentas (chave e macaco)
- 3: pegar o estepe
- 4: suspender o carro com o macaco
- 5: desenroscar os 4 parafusos do pneu furado
- 6: colocar o estepe
- 7: enroscar os 4 parafusos
- 8: baixar o carro com o macaco
- 9: guardar as ferramentas

Níveis de linguagem de programação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

- Programadores, além de expressar e analisar algoritmos, estão preocupados em desenvolver *softwares* que instruem máquinas a desempenhar diversas atividades.

Passar dados de entrada

Editar

Traduzir

Interpretar programas nas máquinas

Processadores de linguagem de programação

Níveis de linguagem de programação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

- As máquinas são guiadas por programas expressos em **código de máquina (machine-code)**
- Um programa *machine-code* é uma sequência de instruções, onde cada instrução é uma cadeia de bits que é interpretada pela máquina para desempenhar a operação definida.

```
0000 0001 0110 1110  
0100 0000 0001 0010  
1100 0000 0000 1101
```



Níveis de linguagem de programação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

- Traduzindo...

0000 0001 0110 1110 - Carregue um dado do endereço de memória 366

0100 0000 0001 0010 - Adicione dois números localizados nos
registradores 1 e 2

1100 0000 0000 1101 - Pule para a instrução 13 se o resultado da
operação anterior for zero



Níveis de linguagem de programação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

- Uma vez escrito, o programa pode ser simplesmente carregado na máquina e executado.
- Claramente, programas ***machine-code*** são extremamente difíceis de serem lidos, escritos e editados.
 - Cadeias de bits e endereços de memória não são intuitivos
- Linguagem de máquina é a única compreendida pelo computador
 - Chaves eletrônicas em dois estados: desligado (0) ou ligado (1)

Níveis de linguagem de programação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

- Desvantagens da linguagem de máquina:
 - Depende da máquina, ou seja, se os hardwares forem diferentes, suas linguagens de máquina serão diferentes
 - É enfadonho escrever programas nesta linguagem e é difícil de encontrar erros
- Para facilitar a vida dos programadores, **notações simbólicas** surgiram...

Níveis de linguagem de programação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

Que tal?

0000 0001 0110 1110 - Carregue um dado do endereço de memória 366

LOAD x

0100 0000 0001 0010 - Adicione dois números localizados nos registradores 1 e 2

ADD R1 R2

1100 0000 0000 1101 - Pule para a instrução 13 se o resultado da operação anterior for zero

JUMPZ h



Níveis de linguagem de programação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

LOAD, ADD e JUMPZ – operações

R1 e R2 – registradores

x – endereço de memória de um dado

h – endereço de memória de uma instrução particular

Esta notação simbólica chama-se **Linguagem Assembly**, ou linguagem de montagem!

Níveis de linguagem de programação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

Linguagem Assembly

- No Assembly, o código binário é substituído por instruções e endereços com símbolos e mnemônicos
- Mesmo que o programador utilize a linguagem assembly, ele ainda está trabalhando no nível de instruções de máquina
 - Instruções bem primitivas, que devem ser lidas individualmente e colocadas na sequência correta
 - O programador pensa nos detalhes dos registradores, *jumps*, etc.

Um exemplo



$$\sqrt{s \times (s - a) \times (s - b) \times (s - c)}, \text{ onde } s = \frac{(a+b+c)}{2}$$

LOAD R1 a; ADD R1 b; ADD R1 c; DIV R1 #2;

LOAD R2 R1;

LOAD R3 R1; SUB R3 a; MULT R2 R3;

LOAD R3 R1; SUB R3 b; MULT R2 R3;

LOAD R3 R1; SUB R3 c; MULT R2 R3;

LOAD R0 R2; CALL sqrt

Níveis de linguagem de programação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

- Linguagens de máquina (assembly) são chamadas **linguagem baixo nível** (*low-level languages*)
 - Forçam os algoritmos a serem expressos em termos de instruções primitivas, a serem executadas diretamente pelo hardware
- Linguagens que usam notação similar à notação matemática são chamadas de **alto nível** (*high-level languages*)
 - Os algoritmos são expressos de forma mais próxima a sua conceptualização na nossa cabeça
 - São portáteis para muitos computadores, permitindo que o programador se concentre na aplicação, e não na especificidade da organização de computadores

Níveis de linguagem de programação



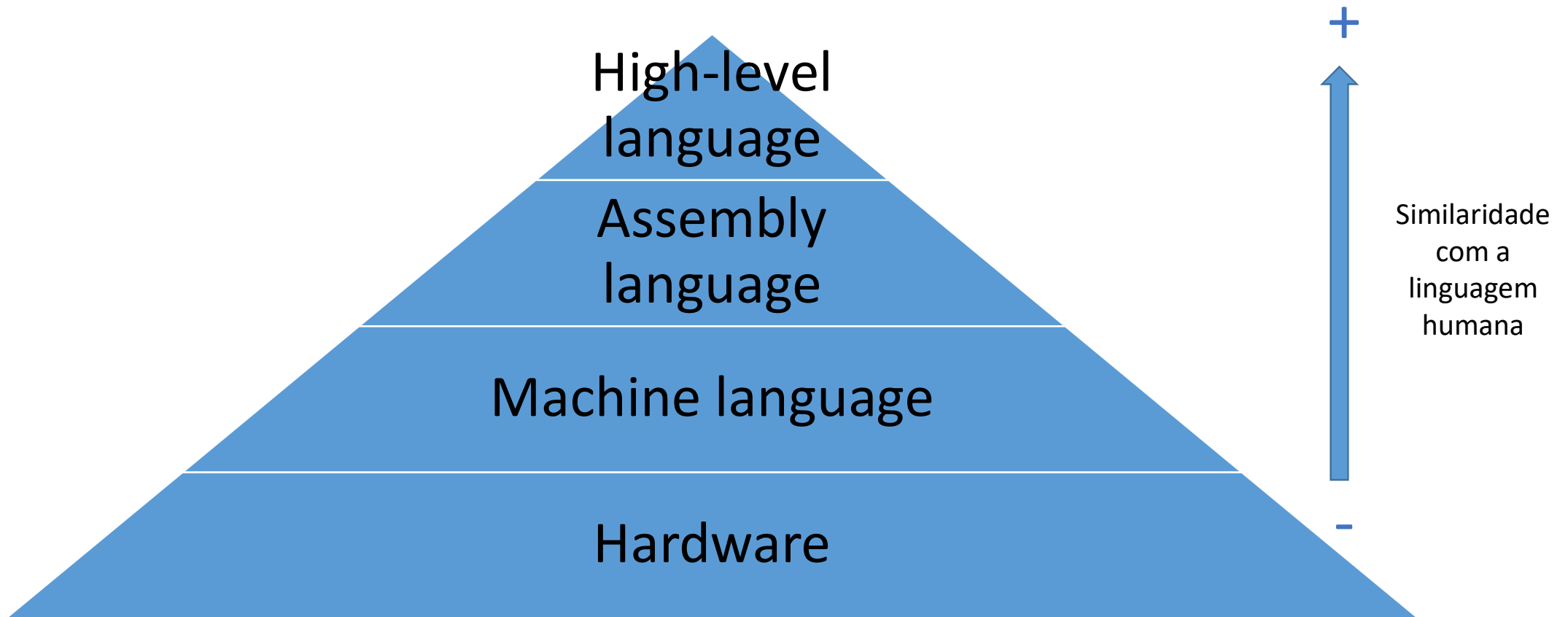
INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

- São conceitos típicos de linguagens **alto-nível** (e suportados de forma rudimentar por linguagens baixo-nível):
 - **Expressões**, utilizadas de forma similar à notação matemática, com operadores $+$, $-$, $*$ e $/$.
 - **Tipos de dados** são manipulados pelos programas, como booleano, caracteres, inteiros, tipos compostos, como arrays. E ainda declarar constantes, variáveis, funções e parâmetros destes tipos.
 - **Estruturas de controle**, com comandos de seleção (*if* e *cases*) e computação iterativa (*while* e *for*)
 - **Declarações** que permitem definir identificadores para denotar constantes, variáveis, procedimentos, funções e tipos
 - **Abstração** com a separação do “o quê” e do “como” deve ser realizado
 - **Encapsulamento** ao agrupar variáveis com operações (algo que é essencial à programação orientada a objetos)

Níveis de linguagem de programação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO



Compilação e Interpretação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

- Tanto as linguagens alto-nível como as baixo-nível precisam ser convertidas para linguagem de máquina
 - Esse processo chama-se ***interpretação*** ou ***compilação***
- Programa em linguagem alto nível -> **programa-fonte**
- Programa em linguagem de máquina -> **programa-objeto**

Compilação e Interpretação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

- **Compilação**

- Compilador traduz o programa-fonte completo para o programa-objeto. Depois do processo o programa é executado.
 - O programa será executado com desempenho máximo
 - Tempo de espera pela compilação

- **Interpretação**

- Processo de traduzir cada linha do programa-fonte na linha correspondente do programa-objeto e executar a linha. Uma por vez.
 - Se houver qualquer erro na tradução e na execução, o processo exibe uma mensagem e o restante é abortado.
 - Produz saídas assim que a primeira instrução é processada
 - Processo lento

Compilação e Interpretação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

Interpretadores

- Quando usar
 - O programador está trabalhando de forma interativa e deseja ver os resultados de cada instrução antes de digitar a próxima (ex.: consultas SQL)
 - O programa será usado uma vez e então descartado e a velocidade de execução não é muito importante
 - As instruções possuem formatos simples e podem ser analisadas facilmente e de forma eficiente
- Execução mais lenta
 - Interpretar uma linguagem de alto nível pode ser até 100 vezes mais lento do que executar a versão compilada
- Não usar quando
 - A performance é muito importante
 - As instruções possuem formatos complicados
- Exemplos: interpretador SQL, shell do Unix e do DOS, Basic...

Compilação e Interpretação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

Interpretive Compiler

Combinação de compilador + interpretador com o objetivo de aproveitar o benefício de ambos processadores da linguagem

- Ideia chave: compilar o código fonte para uma **linguagem intermediária** que deve atender aos seguintes requisitos:
 - Linguagem intermediária entre a linguagem fonte e o código de máquina
 - Instruções em **formato simples** e que podem ser **facilmente e rapidamente analisadas**
 - **Compilação** da linguagem fonte para a linguagem intermediária é **fácil e rápida**
- Ex: JDK (Java)

Compilação e Interpretação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

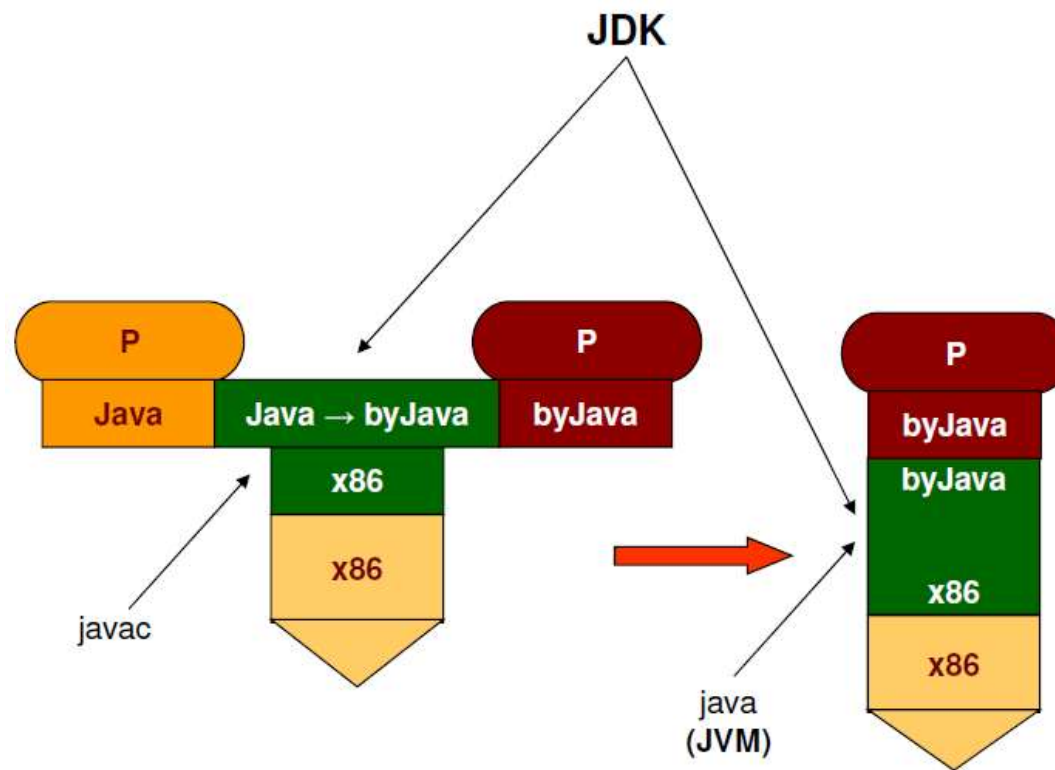
Interpretive Compiler

- **Linguagem Java -> Compilada + Interpretada**
 - **Portabilidade!**
 - Tradução do programa-fonte em objeto em duas etapas:
 - **Etapa 1: Compilação** do programa-fonte em Java para criar o **bytecode (linguagem intermediária)**, o qual é similar à linguagem de máquina mas não é específica para nenhum computador. **Bytecode é um código objeto para a JVM (Java Virtual Machine)**
 - **Etapa 2: Compilação ou Interpretação** do bytecode por qualquer computador que execute o emulador da JVM
 - Ou seja, o computador que executa o bytecode precisa somente de um emulador JVM, e não mais do compilador Java.

Compilação e Interpretação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO



Compilação e Interpretação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO



Java Compiler javac
(Solaris, Linux, or Windows)



Java Bytecode
(.class)



JVM java
Solaris JRE



JVM java
Linux JRE

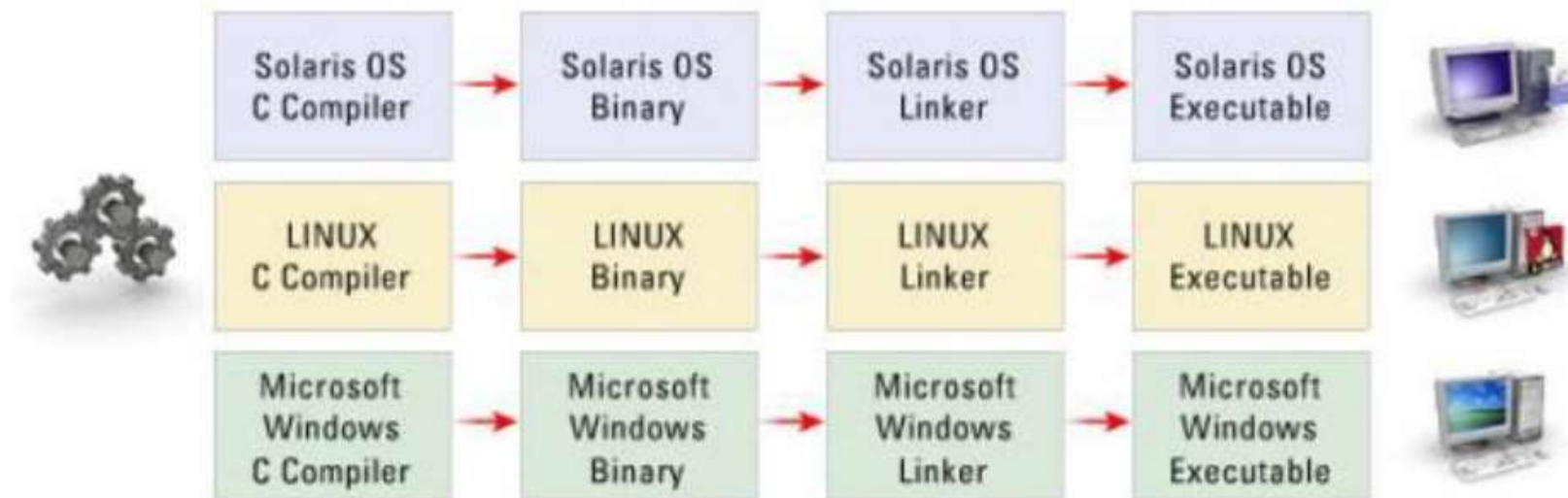


JVM java.exe
Windows JRE

Compilação e Interpretação



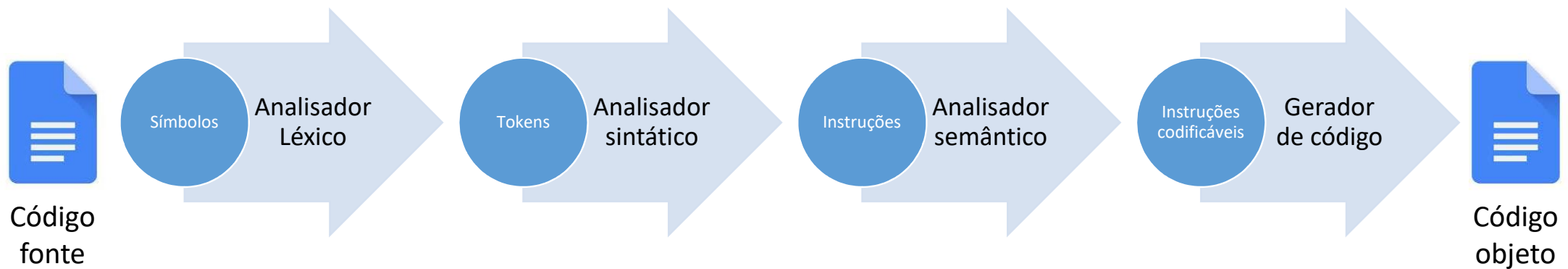
INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO



Processo de Tradução



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

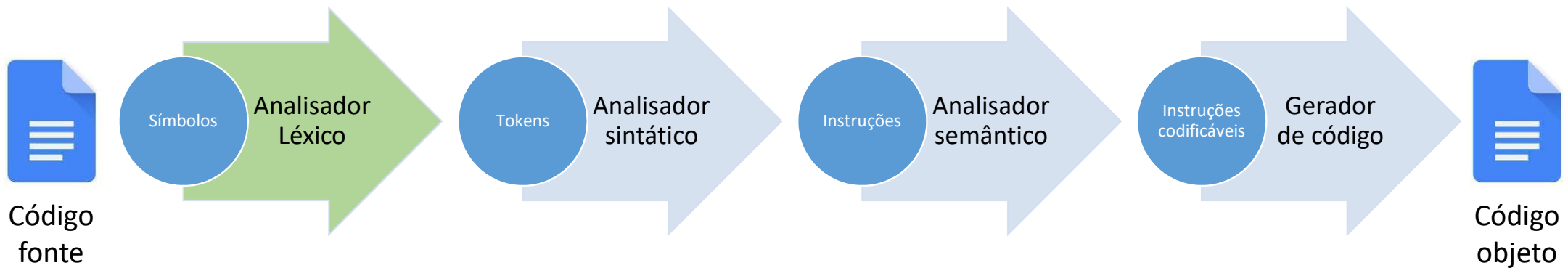


Processo de Tradução



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

Analizador léxico: O código-fonte é lido, símbolo por símbolo, e cria uma lista de tokens da linguagem. Ex.: Os símbolos *w*, *h*, *i*, *l*, *e* são lidos e agrupados no token *while* na linguagem Java.

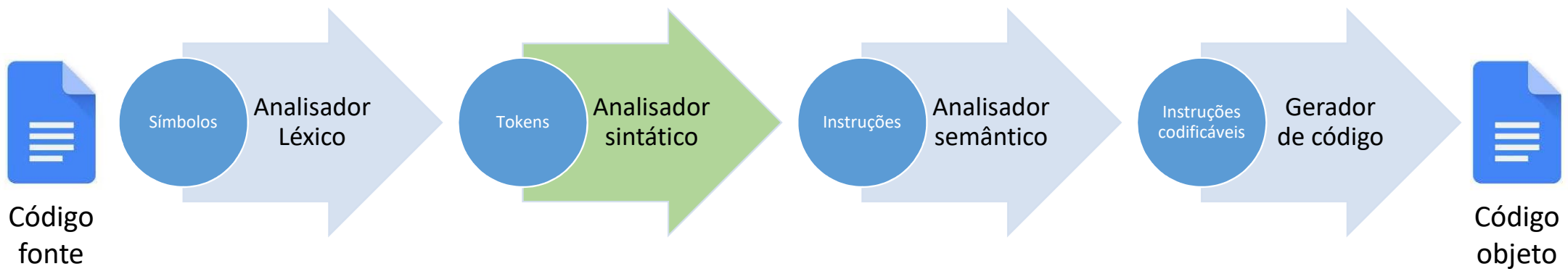


Processo de Tradução



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

Analizador sintático: Analisa um conjunto de tokens, a fim de encontrar instruções de acordo com a estrutura de frases da linguagem. Ex.: Os tokens 'x', '=' e '0' são utilizados para criar o comando de atribuição em Java.

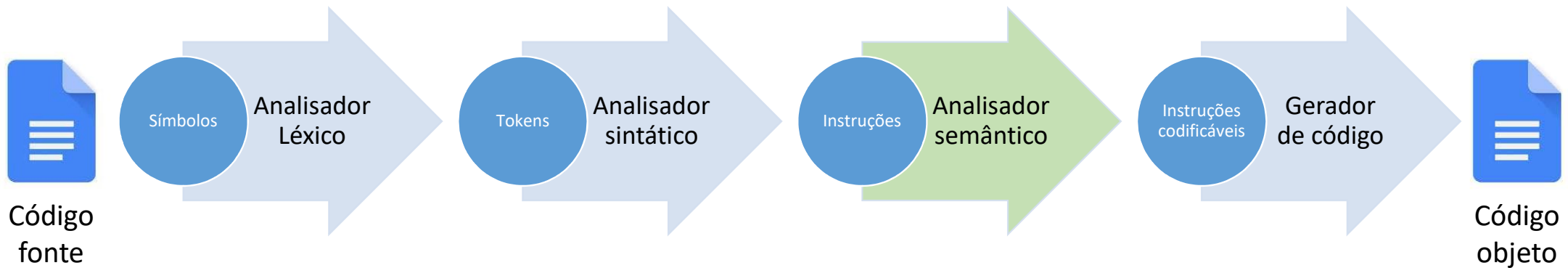


Processo de Tradução



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

Analizador semântico: Verifica se as sentenças criadas pelo analisador sintático não contêm ambiguidade e se estão de acordo com as restrições contextuais da linguagem. Ex.: Verificar se uma variável foi declarada antes de ser utilizada em alguma expressão.

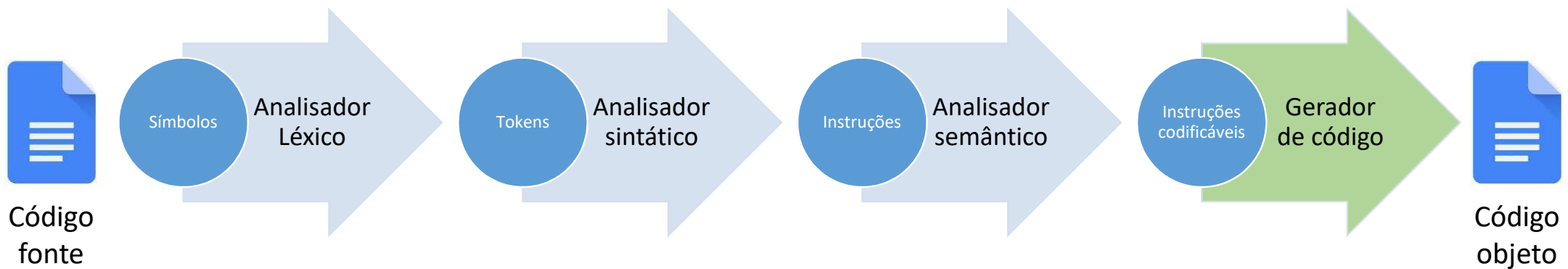


Processo de Tradução



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

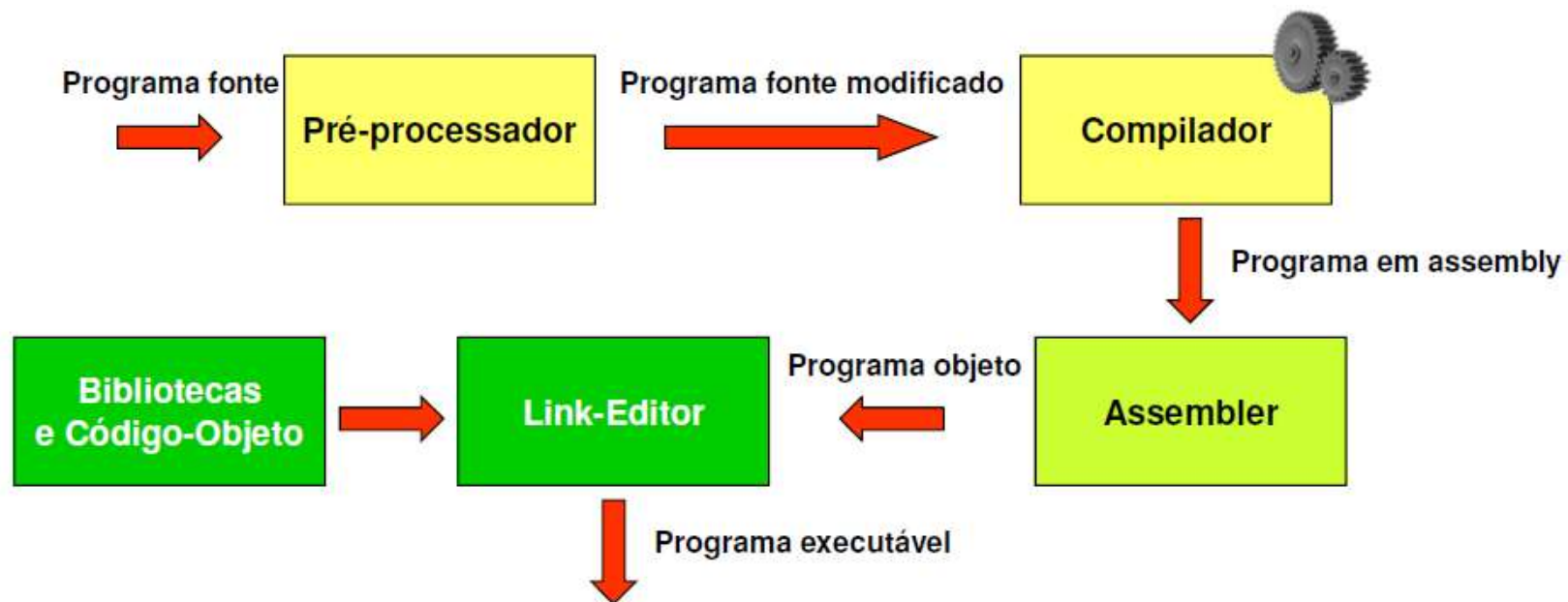
Gerador de código: Cada instrução é convertida em linguagem de máquina para o computador no qual o programa será executado. Pode-se ir além e fazer otimizações na geração de código também, como, por exemplo: eliminar código redundante ou desnecessário.



Processo de compilação



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO



Referências



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

- Watt, D.; Brown, D. Programming Language Processors in Java.
 - Capítulos 1 e 2.
- Forouzan, B.; Mousharraf, F. Fundamentos da Ciência da Computação.
 - Capítulo 9